Lec-31

08/03/26 12:44 PM

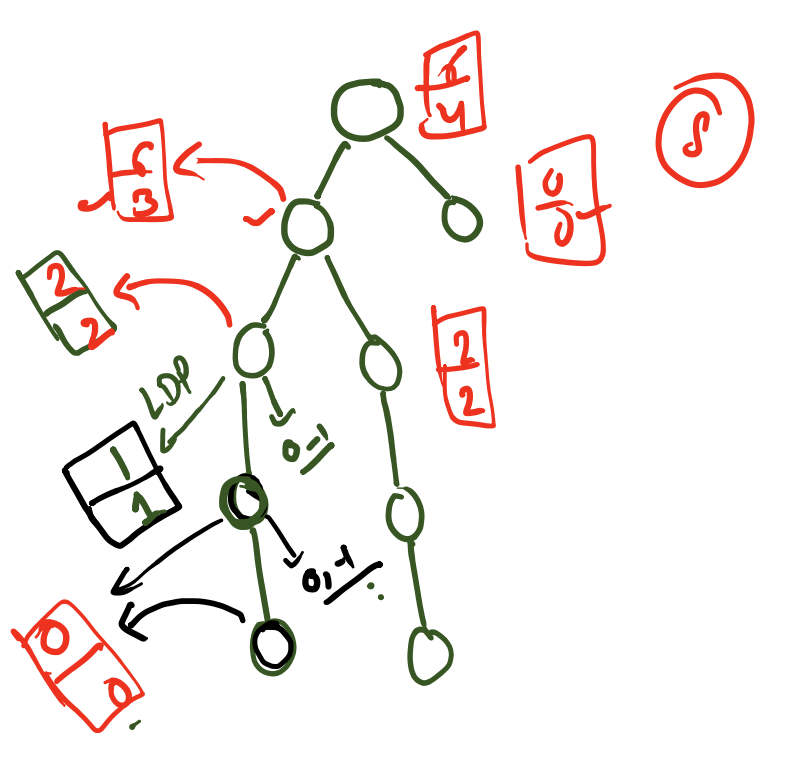
```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }
        DiaPair ldp = diameter(root.left);
        DiaPair rdp = diameter(root.right);
        int sd = ldp.ht + rdp.ht + 2;
        DiaPair sdp = new DiaPair();
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        sdp.d = Math.max(ldp.d, Math.max(rdp.d, sd));
        return sdp;

    }
}
class DiaPair {
    int d = 0;
    int ht = -1;
}
```

0 0 1

-1+-1+2=0

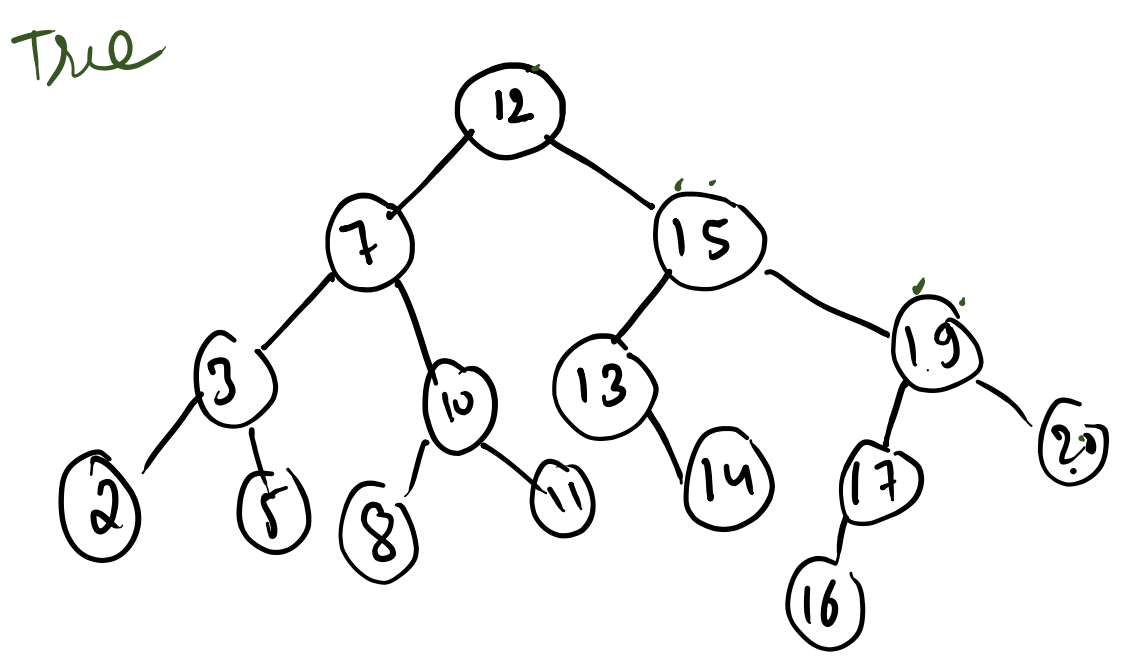
0-1+2=1

1+1=2



Binary Search Tree

Sorted 2 3 5 7 8 10 11 12 13 14 15 16 17 19 20

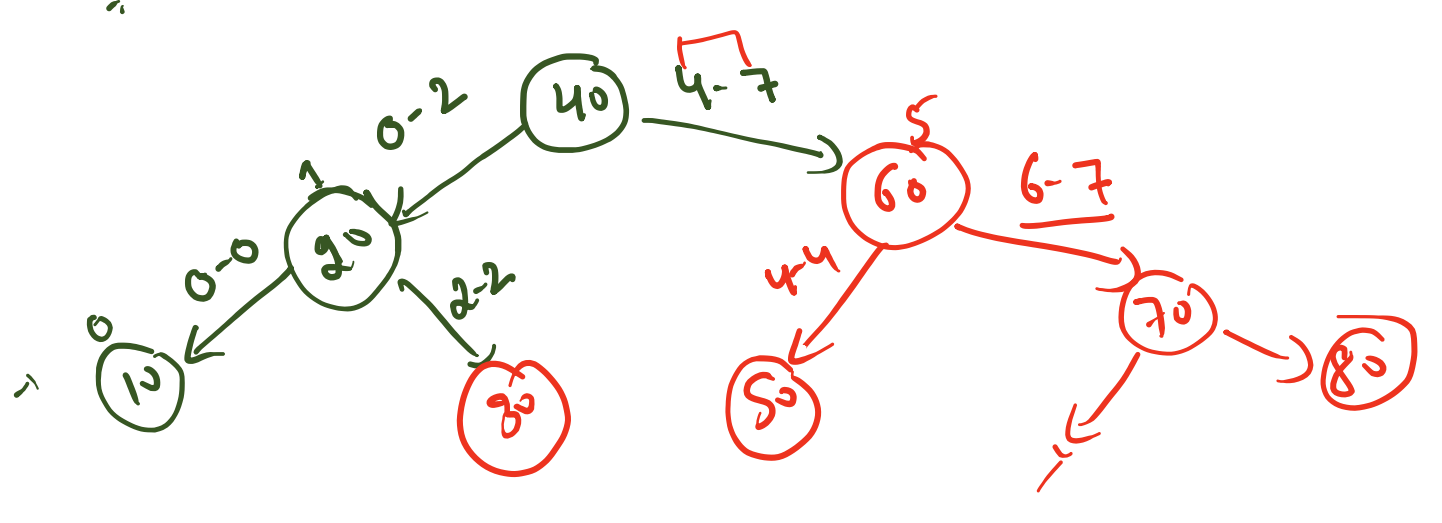


R = max(m1, ...))

m-x (R1, ...

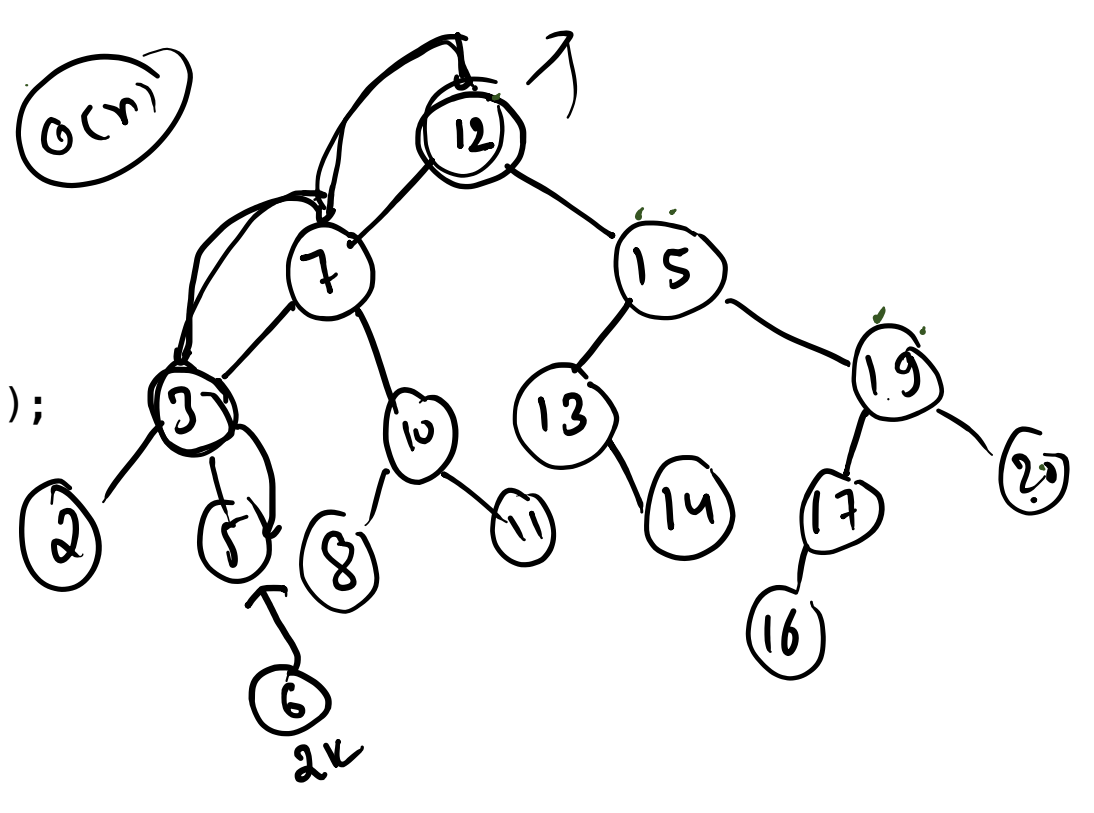
[10, 20, 30, 40, 50, 60, 70, 80] → inorder

0 1 2 3 4 5 6 7

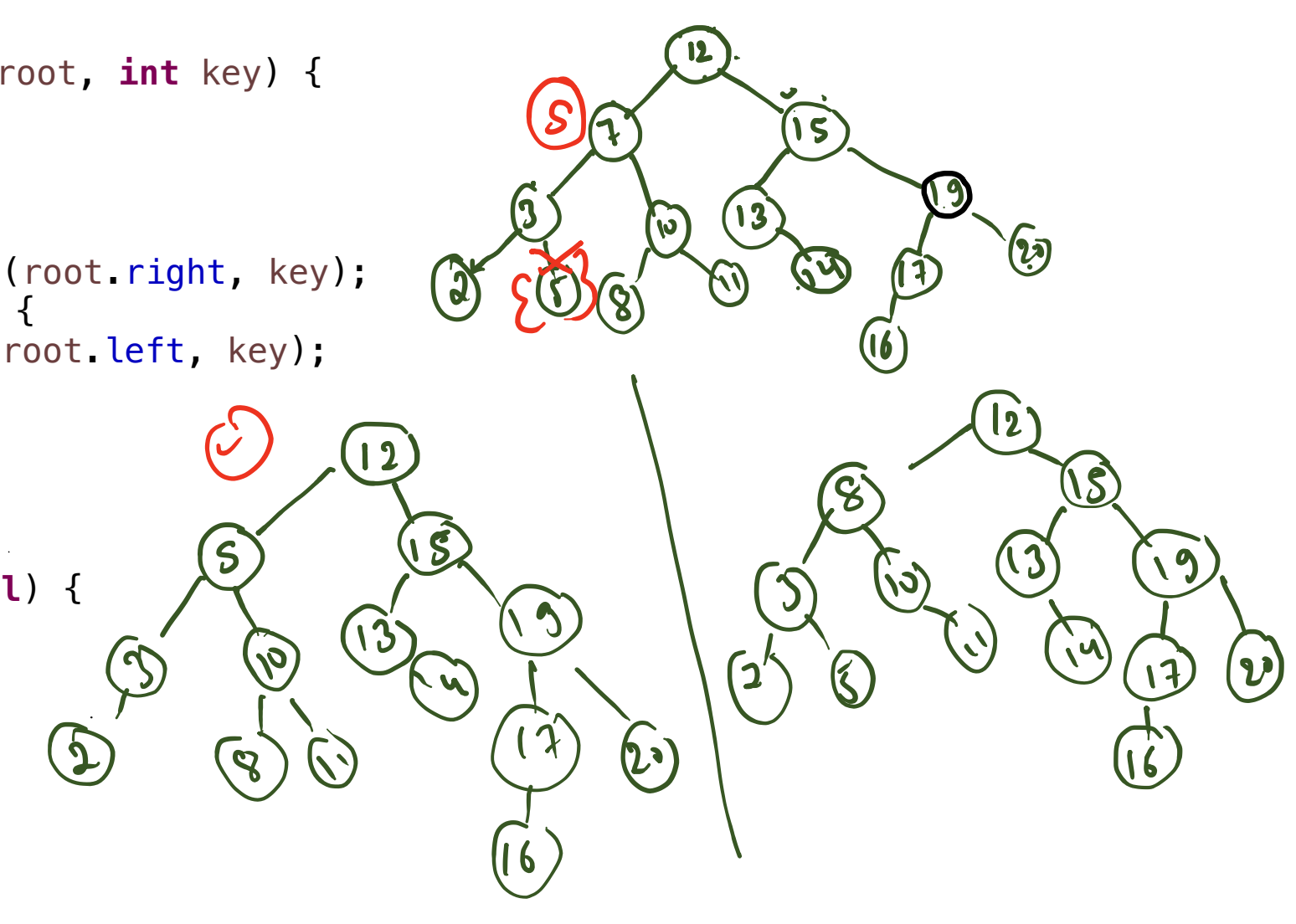


```
public TreeNode insertIntoBST(TreeNode root, int val) {
        if (root == null) {
            return new TreeNode(val);
        }

        if (root.val < val) {
            root.right = insertIntoBST(root.right, val);
        } else {
            root.left = insertIntoBST(root.left, val);
        }
        return root;
    }
```

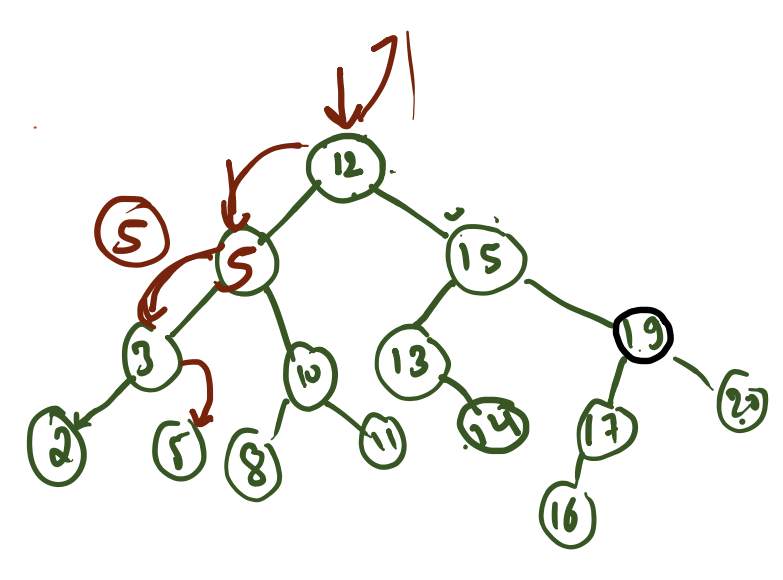
O(n)



```
public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) {
            return root;
        }
        if (root.val < key) {
            root.right = deleteNode(root.right, key);
        } else if (root.val > key) {
            root.left = deleteNode(root.left, key);
        } else {
            // 1 chlid
            if(root.left==null) {
                return root.right;
            }
            else if(root.right==null) {
                return root.left;
            }

        }

        return root;

    }
```

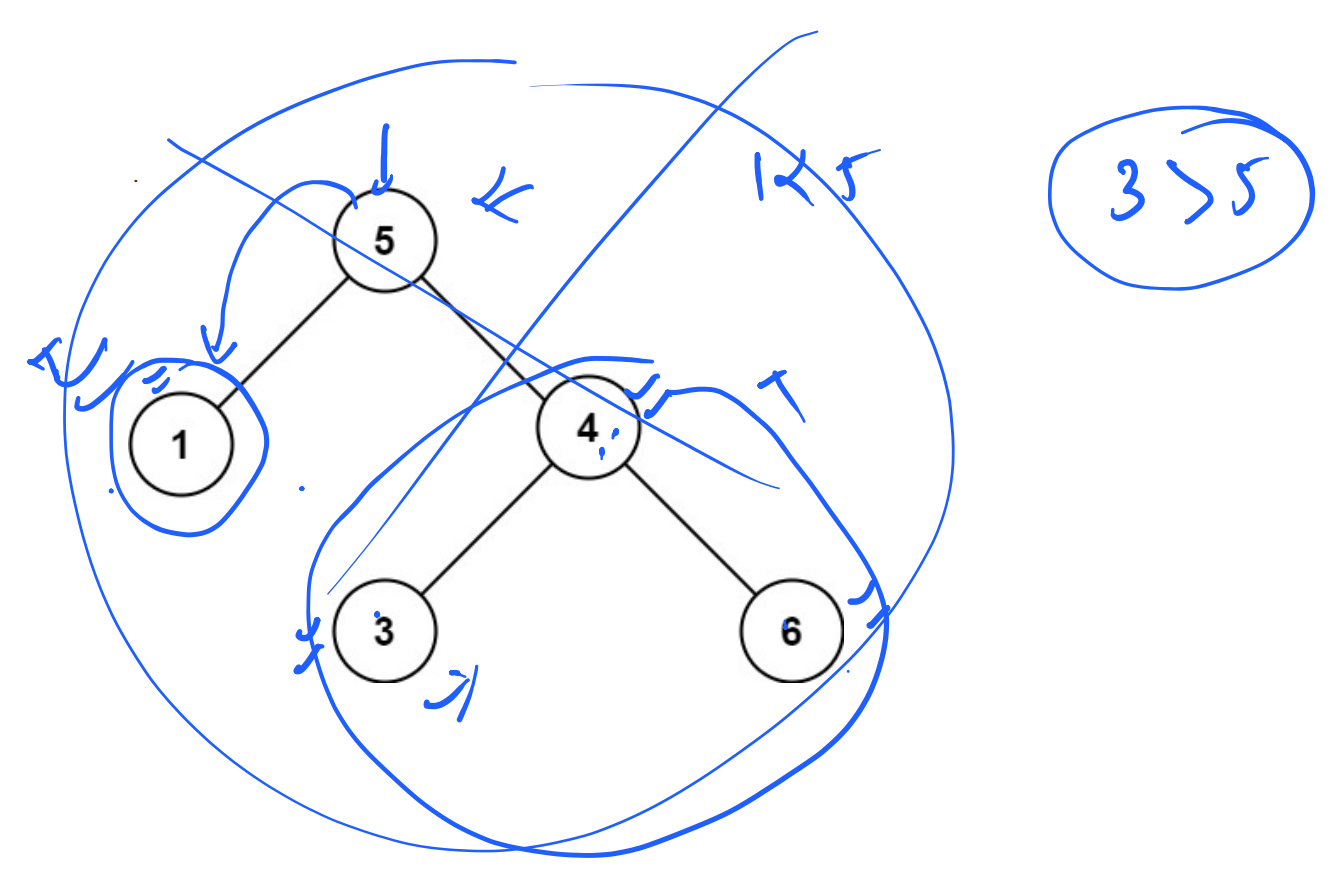


```
public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) {
            return root;
        }
        if (root.val < key) {
            root.right = deleteNode(root.right, key);
        } else if (root.val > key) {
            root.left = deleteNode(root.left, key);
        } else {
            //0 or 1 chlid
            if(root.left==null) {
                return root.right;
            }
            else if(root.right==null) {
                return root.left;
            }
            else {// both child
                int max=max(root.left);
                root.left=deleteNode(root, max);
                root.val=max;

            }

        }
        return root;
    }
```



12

5 15



Class BstPair2

boolean isbst = True

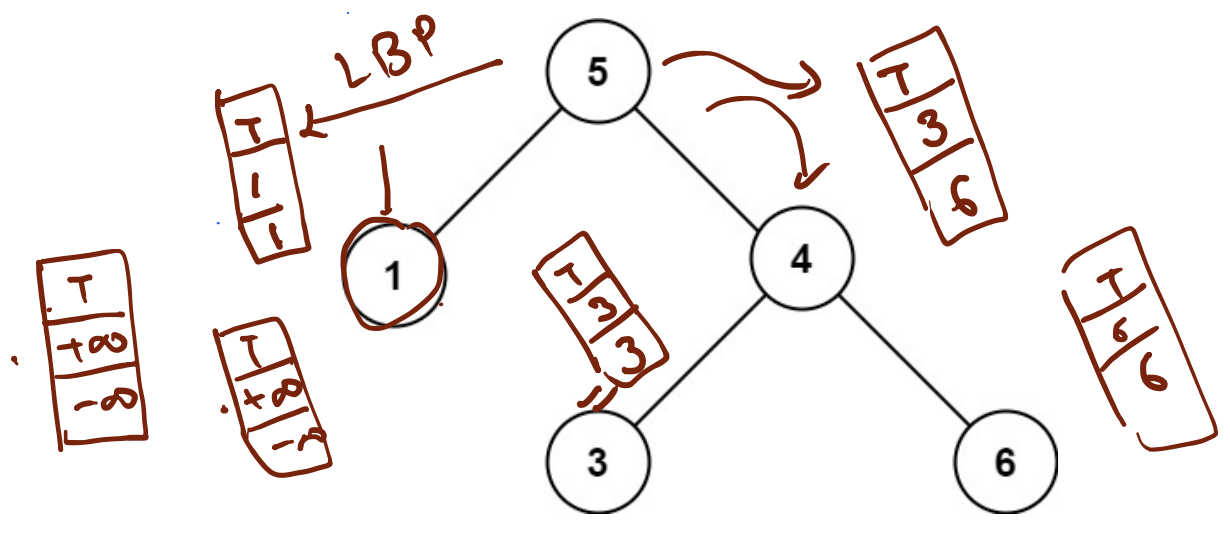
min = ∞

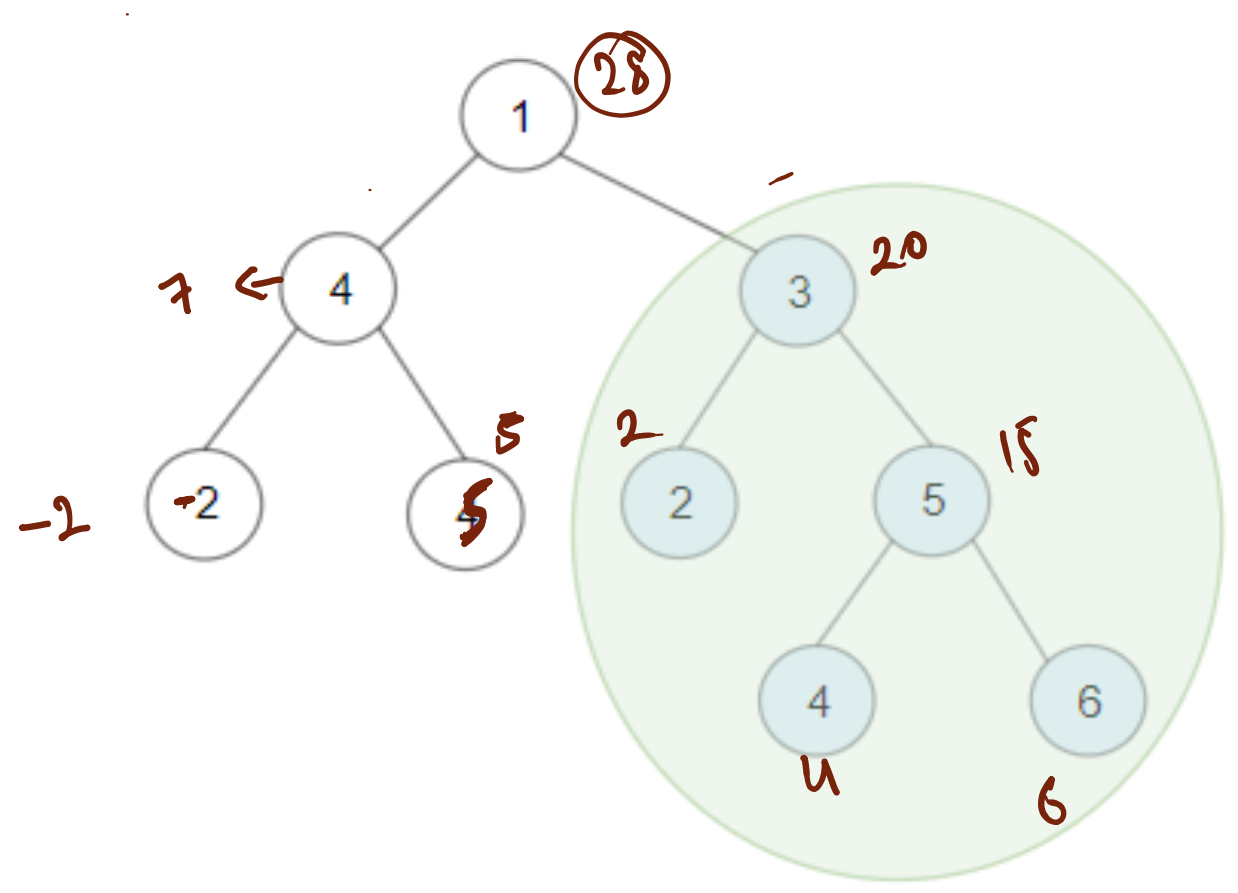
max = -∞

```
public BstPair ValidBST(TreeNode root) {
        if (root == null) {
            return new BstPair();
        }
        BstPair lbp = ValidBST(root.left);
        BstPair rbp = ValidBST(root.right);
        BstPair sbp = new BstPair();
        sbp.min = Math.min(lbp.min, Math.min(rbp.min, root.val));
        sbp.max = Math.max(lbp.max, Math.max(rbp.max, root.val));
        if (lbp.isbst && rbp.isbst && lbp.max < root.val && rbp.min > root.val) {
            sbp.isbst = true;
        } else {
            sbp.isbst = false;
        }
        return sbp;
    }
```

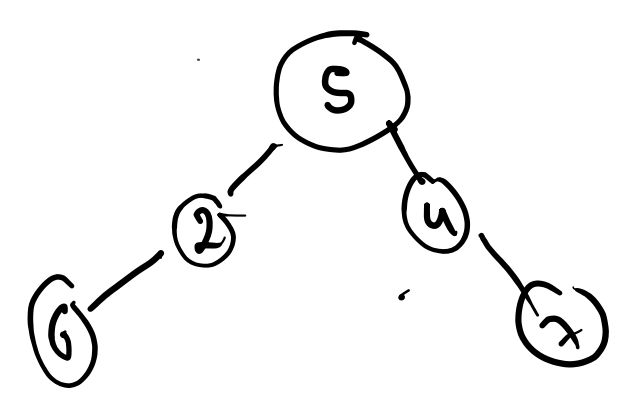
T -min max

F 1 6





5 2 4 6 -1 -1 7 -1 -1 -1 -1



5 2 4 6 7

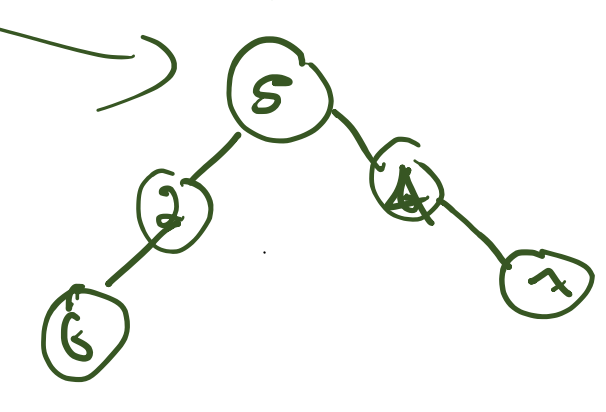
5

```
Queue<Node> q = new LinkedList<>();
        int item = sc.nextInt();
        Node nn = new Node();
        nn.val = item;
        root = nn;
        q.add(nn);
```

⇒ 5

```
private void CreateTree() {
        // TODO Auto-generated method stub
        Queue<Node> q = new LinkedList<>();
        int item = sc.nextInt();
        Node nn = new Node();
        nn.val = item;
        root = nn;
        q.add(nn);
        while (!q.isEmpty()) {
            // remove
            Node n = q.poll();
            int c1 = sc.nextInt();
            int c2 = sc.nextInt();
            if (c1 != -1) {
                Node node = new Node();
                node.val = c1;
                n.left = node;
                q.add(node);
            }
            if (c2 != -1) {
                Node node = new Node();
                node.val = c2;
                n.right = node;
                q.add(node);
            }

        }
    }
```

5 2 4 6 7

5 2 4 6 -1 -1 7 -1 -1 -1 -1